COS30049 – Computing Technology Innovation Project

Week 12 - Software Testing

Ningran Li
Ningran.li@swin.edu.au

Acknowledgement of Country

We respectfully acknowledge the Wurundjeri People of the Kulin Nation, who are the Traditional Owners of the land on which Swinburne's Australian campuses are located in Melbourne's east and outer-east, and pay our respect to their Elders past, present and emerging.

We are honoured to recognise our connection to Wurundjeri Country, history, culture, and spirituality through these locations, and strive to ensure that we operate in a manner that respects and honours the Elders and Ancestors of these lands.

We also respectfully acknowledge Swinburne's Aboriginal and Torres Strait Islander staff, students, alumni, partners and visitors.

We also acknowledge and respect the Traditional Owners of lands across Australia, their Elders, Ancestors, cultures, and heritage, and recognise the continuing sovereignties of all Aboriginal and Torres Strait Islander Nations.



Objectives of Today

1. Master core concepts and types of software testing
2. Apply key testing methodologies and strategies in practice
3. Design, execute, and automate effective test cases
4. Integrate testing with project management processes
5. Adapt testing approaches for ML models and web applications



Image from [Pinterest]

- **Introduction to Software Testing**
- Types of Software Testing
- Test Planning and Strategy
- Test Case Design Techniques
- Test Execution and Reporting
- Automated Testing

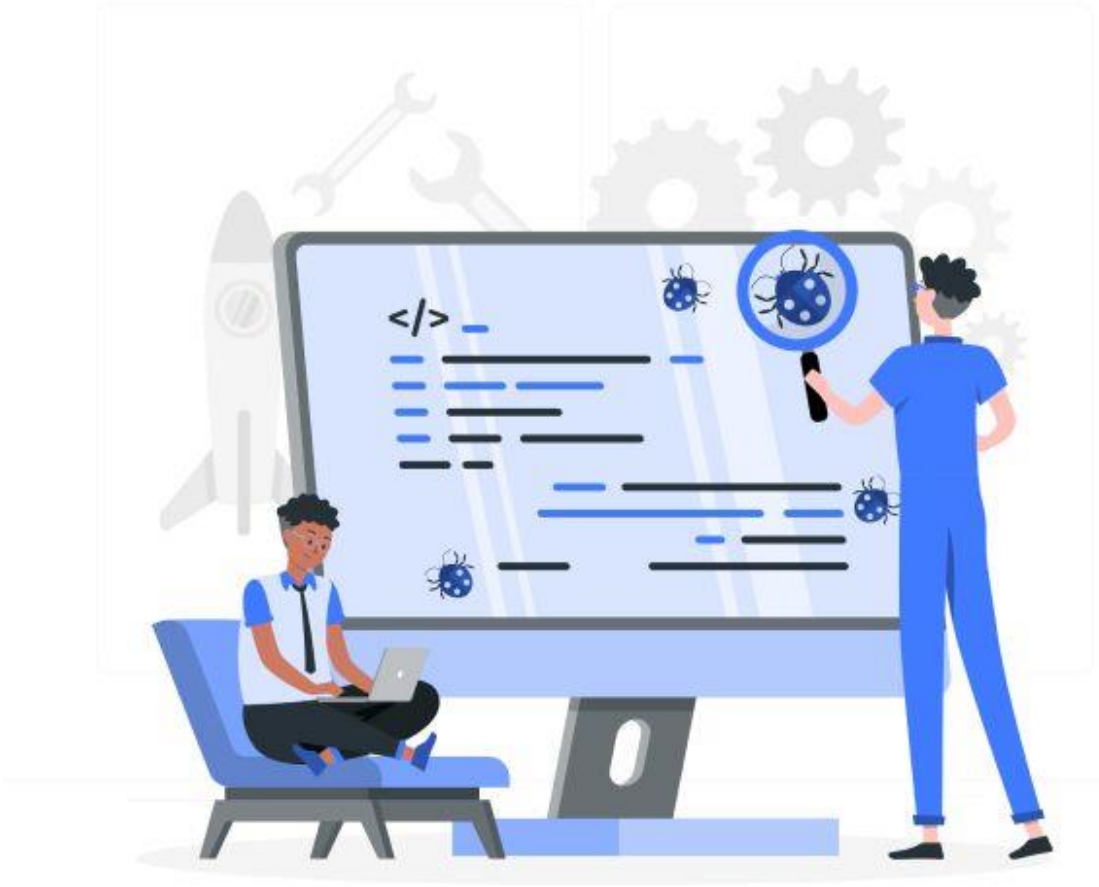


Image from [Pinterest]

What is Software Testing?

Definition:

Systematic process of evaluating software to identify defects and assess quality

Key aspects:

Verification: Does the software meet specified requirements?

Validation: Does the software meet user needs?

Goal:

Ensure software reliability, functionality, and performance

SWIN BUR NE SWINBURNE UNIVERSITY OF TECHNOLOGY

Importance in Software Development Lifecycle

Quality assurance

Ensures product meets quality standards

Risk mitigation

Identifies and addresses potential issues early

Cost-effectiveness

Reduces long-term maintenance costs

User satisfaction

Improves user experience and trust

Compliance

Ensures adherence to industry standards and regulations



Image from [Pinterest]

Testing in Project Management

Testing in project phases:

- Planning: Defining test strategy and resources
- Execution: Implementing tests alongside development
- Monitoring and Controlling: Tracking defects and test progress
- Closing: Final acceptance testing and sign-off

Impact on project success:

- Reduces project risks
- Improves product quality
- Aids in meeting project deadlines and budgets

SWIN BUR NE SWINBURNE UNIVERSITY OF TECHNOLOGY

- Introduction to Software Testing
- **Types of Software Testing**
- Test Planning and Strategy
- Test Case Design Techniques
- Test Execution and Reporting
- Automated Testing

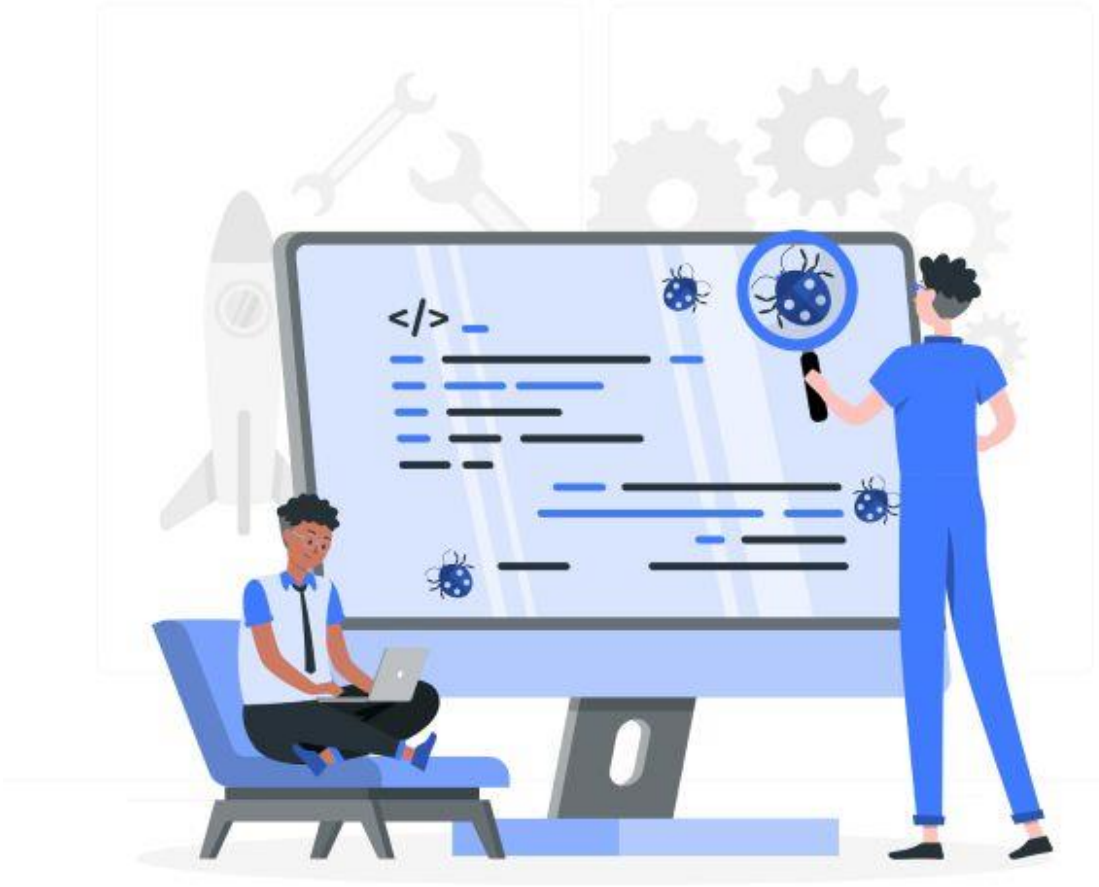


Image from [Pinterest]

Overview of Testing Types

Functional Testing:

Verifies software functions as specified

Non-Functional Testing:

Assesses system properties (performance, usability, etc.)

Structural Testing:

Examines internal structure of software

(e.g. code coverage analysis)

Change-related Testing:

Focuses on changes in software



Image from [Pinterest]

Level of Functional Testing

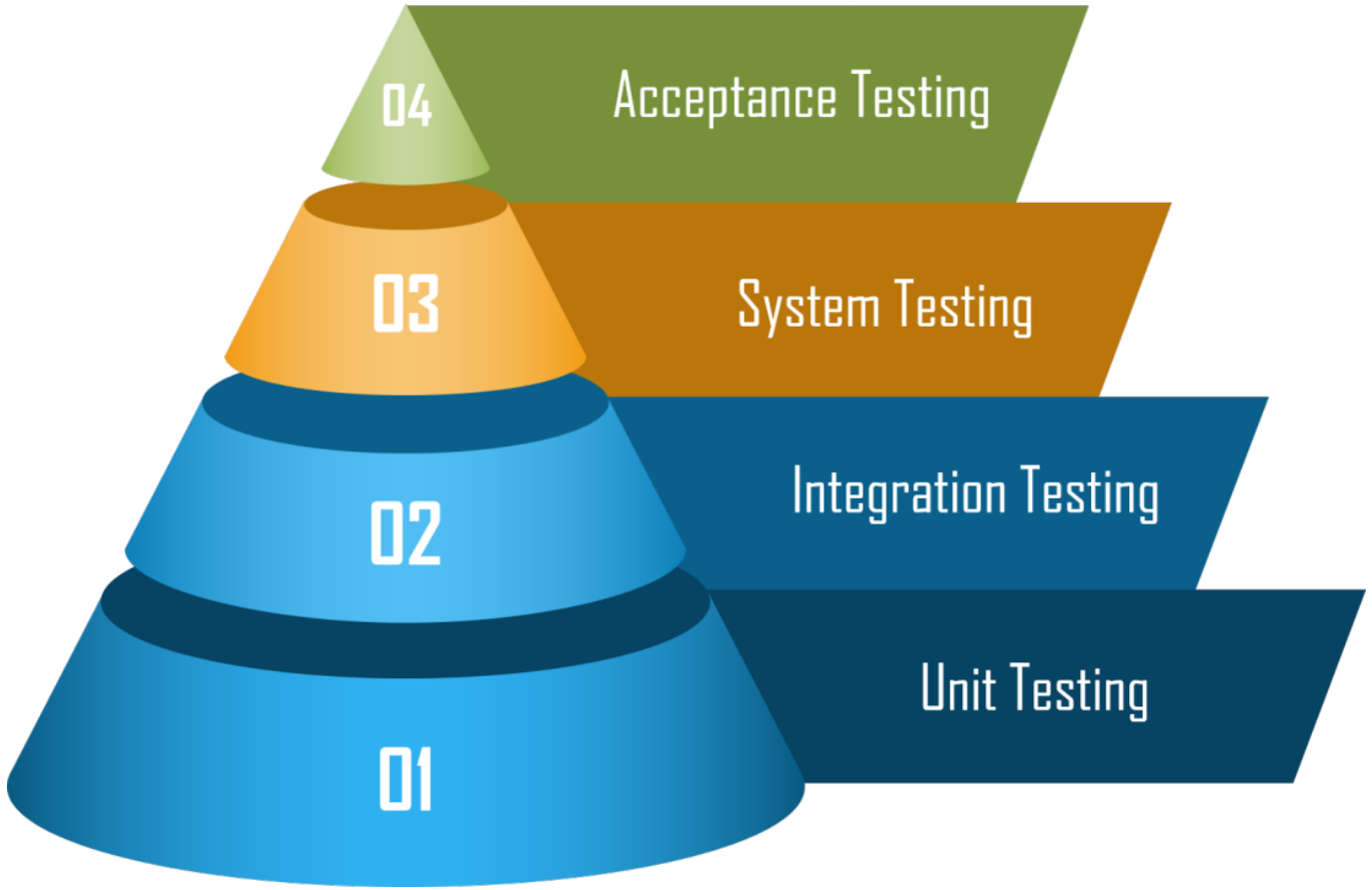


Image from [Source]

Functional Testing - Unit Testing

Definition :

Testing individual components or functions

Characteristics:

Isolated: Tests single units independently

Automated: Often uses frameworks like JUnit, NUnit

Fast: Quick execution and feedback

Benefits :

Early bug detection

Facilitates code refactoring

Improves code quality and design

Example: Testing a function that calculates factorial

SWIN BUR NE SWINBURNE UNIVERSITY OF TECHNOLOGY

Functional Testing - Integration Testing

Definition :

Testing interaction between integrated units/modules

Types:

Big Bang: All modules tested together

Incremental: Top-down, Bottom-up, or Sandwich approach

Focus areas:

Data flow between modules

Functionality of combined modules

Interface defects

Benefits :

Verifies subsystem interactions

Identifies interface-related bugs

Example: Testing API interactions in a web application

SWIN BUR NE SWINBURNE UNIVERSITY OF TECHNOLOGY

Functional Testing - System Testing

Definition :

Testing the complete, integrated software system

Characteristics:

Black-box testing: Focuses on inputs and outputs

Covers functional and non-functional requirements

Simulates real-world usage scenarios

Key aspects:

End-to-end testing | Regression testing | Usability testing

Benefits :

Validates system meets specified requirements

Identifies system-level issues

Example: Testing a complete e-commerce website functionality

SWINBURNE UNIVERSITY OF TECHNOLOGY

Functional Testing - Acceptance Testing

Definition :

Determining if the system satisfies acceptance criteria

Types:

User Acceptance Testing (UAT)

Alpha and Beta testing

Contract Acceptance Testing

Key focus:

Business requirements fulfillment, User satisfaction, Operational readiness

Benefits :

Ensures system meets business needs and Increases user confidence

Final validation before release

Example: Client testing a custom software solution before final approval

SWIN BUR NE SWINBURNE UNIVERSITY OF TECHNOLOGY

Non-Functional Testing



Image from [Headspin]

Non-Functional Testing - Performance Testing

Definition :

Evaluating system behaviour under various conditions to ensure it meets requirements

Types:

Load Testing | Stress Testing: System behaviour under expected/extreme load

Endurance Testing (Soak Testing): System performance over extended periods

Spike Testing: System response to sudden large spikes in load

Key metrics:

Response time | Throughput | Resource utilization | Concurrency

Benefits :

Identifies performance bottlenecks before production deployment

Ensures system meets performance SLAs (Service Level Agreements)

Helps in capacity planning and resource allocation

Example: Testing an e-commerce website's performance during a flash sale event

SWIN BUR NE SWINBURNE UNIVERSITY OF TECHNOLOGY

Non-Functional Testing - Scalability Testing

Definition :

Evaluating the system's ability to scale up or scale out to handle increased load

Types:

Vertical Scaling (Scale Up): Adding more resources to a single node

Horizontal Scaling (Scale Out): Adding more nodes to the system

Key metrics:

Resource utilization | Response time as load increases | Cost-performance ratio

Breakpoint: Where performance degrades unacceptably

Benefits :

Ensures system can handle future growth

Aids in capacity planning and infrastructure decisions

Identifies architectural limitations early

Example: Testing a social platform's ability to handle a 10x increase in user base over 6 months

Non-Functional Testing - Security Testing

Definition :

Identifying vulnerabilities in the system

Focus areas:

Authentication and Authorization | Data Encryption

SQL Injection | Cross-Site Scripting (XSS) | Denial of Service (DoS)

Techniques:

Penetration Testing | Vulnerability Scanning | Risk Assessment

Tools: OWASP ZAP, Nessus, Metasploit

Benefits :

Protects sensitive data

Prevents security breaches

Ensures compliance with security standards

Example: Testing a banking application for potential security vulnerabilities

SWIN BUR NE SWINBURNE UNIVERSITY OF TECHNOLOGY

Non-Functional Testing - Usability Testing

Definition :

Evaluating how user-friendly and efficient the software is from the end-user's perspective

Key Aspects:

Learnability: How easy is it for users to accomplish basic tasks the first time?

Efficiency: How quickly can users perform tasks once they have learned the design?

Memorability: After a period of not using it, how easily can they reestablish proficiency?

Errors: How many errors do users make, how severe are these errors, and how easily can they recover from the errors?

Satisfaction: How pleasant is it to use the design?

Benefits :

Improves user satisfaction and adoption rates | Enhances brand perception and loyalty

Reduces training and support costs | Increases productivity and efficiency

Example: Conducting usability tests on a new mobile banking app to ensure intuitive navigation and easy transaction processes for users of all age groups

SWIN BUR NE SWINBURNE UNIVERSITY OF TECHNOLOGY

Non-Functional Testing - Compatibility Testing

Definition :

Ensuring software works correctly across different environments, devices, and configurations

Key Aspects :

Functionality: Does the software work as expected in all environments?

UI/UX Consistency: Is the user interface consistent across platforms?

Performance: Does the software maintain performance standards in different environments?

Installation and Upgrade: Does the software install and upgrade correctly on all supported systems?

Benefits :

Broadens potential user base

Ensures consistent user experience across platforms

Reduces post-release issues and support costs

Improves market reach and competitiveness

Example: Testing a banking application for potential security vulnerabilities

SWIN BUR NE SWINBURNE UNIVERSITY OF TECHNOLOGY

- Introduction to Software Testing
- Types of Software Testing
- **Test Planning and Strategy**
- Test Case Design Techniques
- Test Execution and Reporting
- Automated Testing

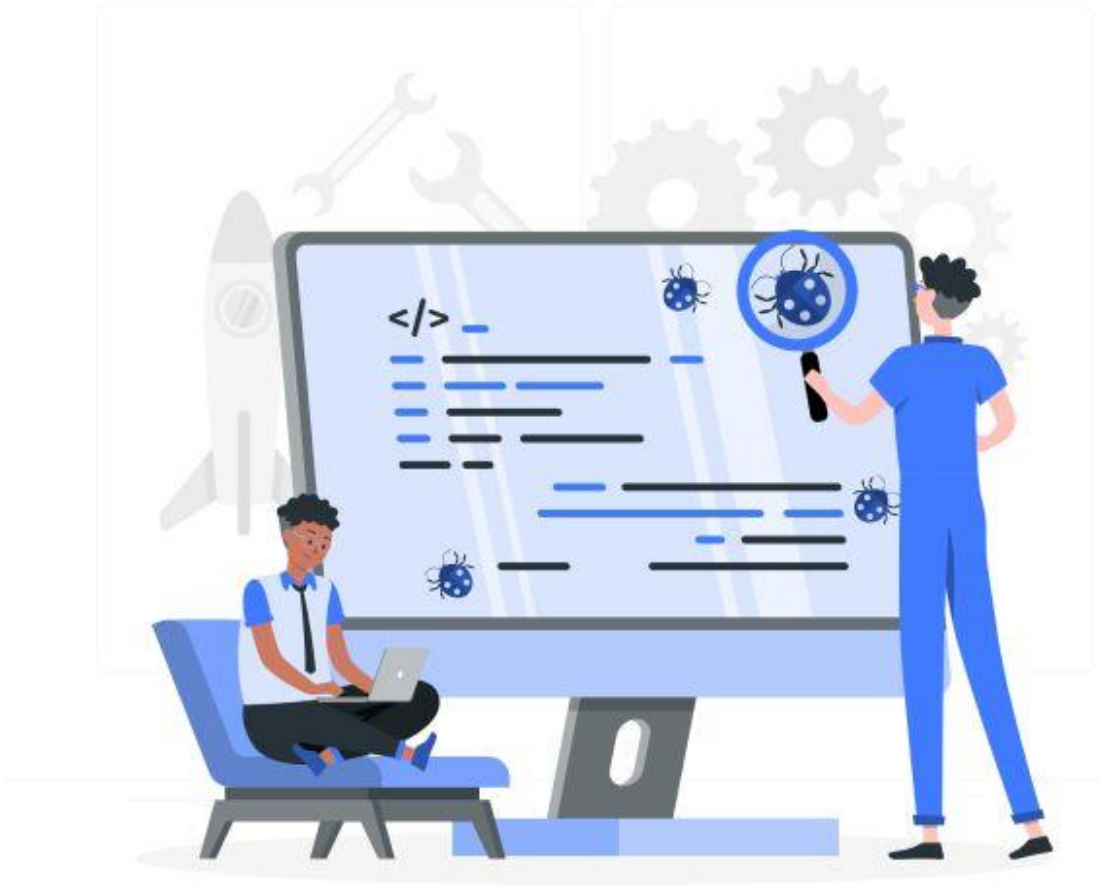


Image from [Pinterest]

Test Plan

Purpose and scope of testing

Test objectives and targets

Testing methodology and approach

Test environment and tools

Test deliverables

Entry and exit criteria

Risk assessment and mitigation strategies

Roles and responsibilities

Schedule and milestones

Defect tracking and reporting procedures

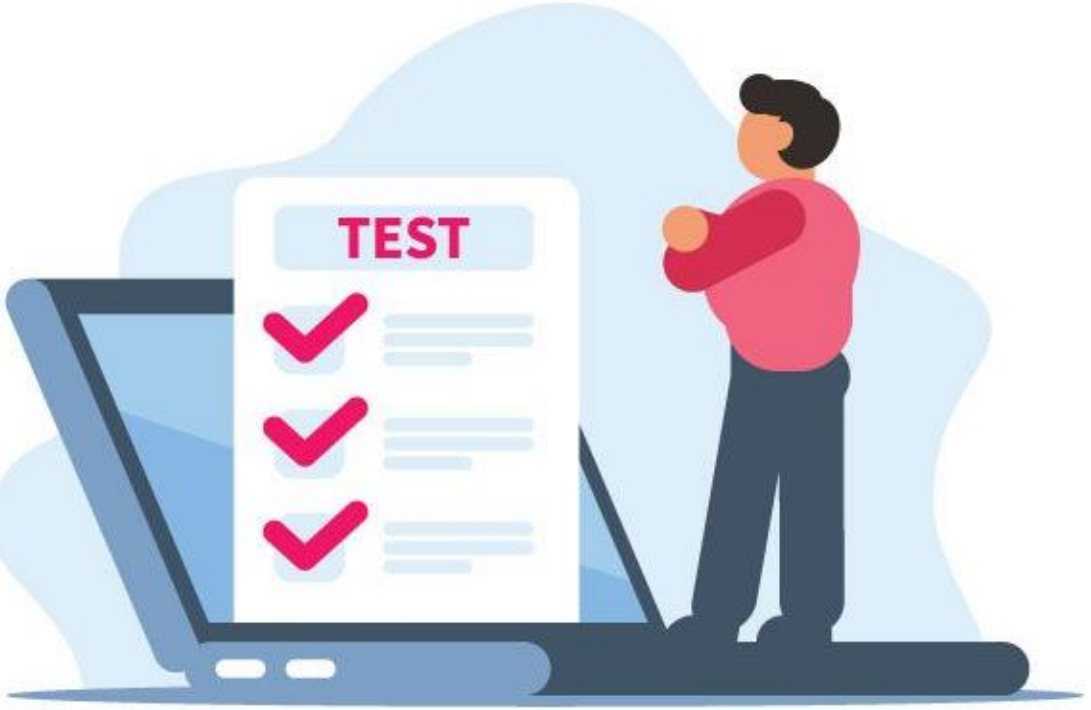


Image from [Pinterest]

Test Strategy Development

- Types of testing to be performed
- Levels of testing (unit, integration, system, acceptance)
- Test data requirements
- Test coverage criteria
- Test prioritization and effort allocation
- Automation strategy
- Configuration management approach
- Metrics for measuring test effectiveness
- Communication and reporting strategy
- Training needs for the testing team

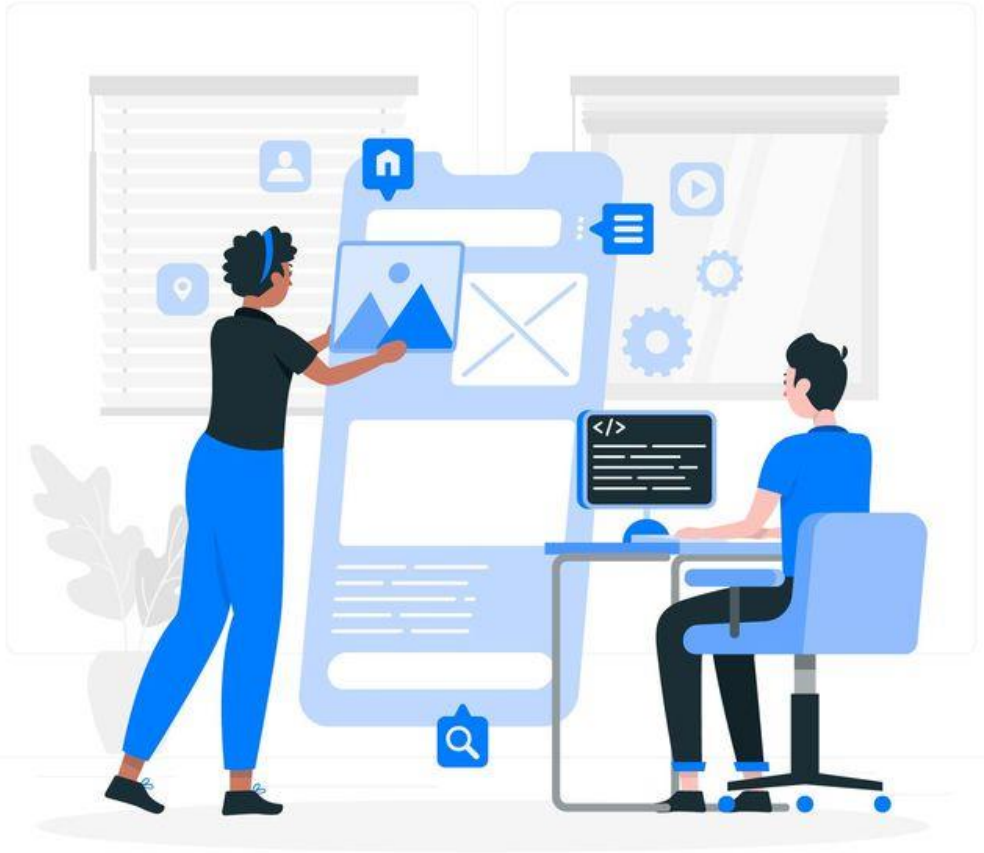


Image from [Pinterest]

Risk-Based Testing Approach

Definition :

Prioritizing testing efforts based on potential risks

Steps:

1. Identify potential risks
2. Assess likelihood and impact of each risk
3. Prioritize risks
4. Develop test cases focused on high-risk areas
5. Allocate resources based on risk priority

Benefits :

Efficient use of testing resources

Focus on critical functionality

Early identification of major issues



Image from [Pinterest]

Example: Prioritizing security testing for a financial application's transaction module

Have a Break and Let's continue with Lecture 02



Image from [Pinterest]

SWINBURNE UNIVERSITY OF TECHNOLOGY